SAP ABAP — Exercise 12

Read Data from a Database Table

Unit 4 — Reading Data from the Database

Purpose

In this exercise, the local class lcl_connection is extended with two private attributes for departure and destination airports. The constructor then reads those values from the database table /DMO/CONNECTION using Open SQL SELECT SINGLE.

What you will learn

- Private instance attributes
- Reading one database row with SELECT SINGLE
- FIELDS and WHERE clauses
- Using @ for ABAP host variables
- Writing selected database values into class attributes with INTO
- Checking SY-SUBRC after SELECT
- Raising CX_ABAP_INVALID_VALUE when no record is found
- Verifying the result in the ABAP Console and debugger

Exercise details

Template	/LRN/CL_S4D400_CLS_CONSTRUCTOR
Suggested class	ZCL_##_SELECT
Solution	/LRN/CL_S4D400_DBS_SELECT
Main database table	/DMO/CONNECTION

Task 1 — Copy Template

You can copy /LRN/CL_S4D400_CLS_CONSTRUCTOR into your own package, or continue from the completed Exercise 11 class. Continuing from Exercise 11 is acceptable because the constructor, private carrier/connection attributes, get_output( ), and conn_counter are already present.

Suggested name:

```
ZCL_##_SELECT
```

For your practice, the existing ZCL_9309_LOCAL_CLASS was extended.

Task 2 — Declare Additional Attributes

The object already knows the carrier and connection. We now add two pieces of internal state: the airport where the flight starts and the airport where it ends.

Step 2.1 — Add the private attributes

```
PRIVATE SECTION.

  DATA carrier_id      TYPE /dmo/carrier_id.
  DATA connection_id   TYPE /dmo/connection_id.
  DATA airport_from_id TYPE /dmo/airport_from_id.
  DATA airport_to_id   TYPE /dmo/airport_to_id.

ENDCLASS.
```

Why private? These values belong to the object and should be controlled by the class. External code should not directly overwrite them.

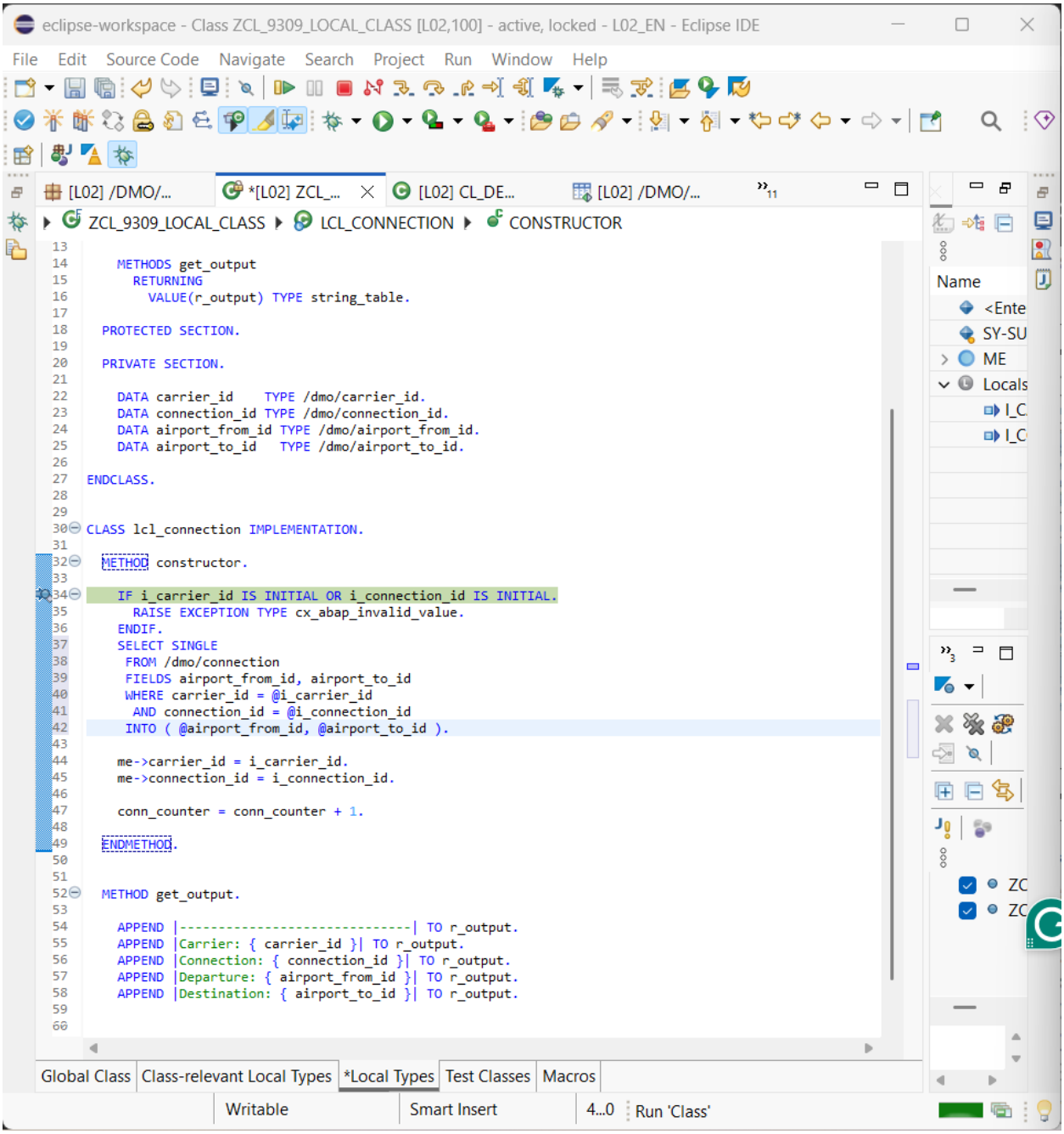
Step 2.2 — Display the new values

```
METHOD get_output.

  APPEND |-----------------------------| TO r_output.
  APPEND |Carrier: { carrier_id }| TO r_output.
  APPEND |Connection: { connection_id }| TO r_output.
  APPEND |Departure: { airport_from_id }| TO r_output.
  APPEND |Destination: { airport_to_id }| TO r_output.

ENDMETHOD.
```

At this stage the new attributes are empty because no database read has been done yet. Seeing blank Departure and Destination before Task 4 is therefore normal.



Screenshot 1 — Your local class after adding the airport attributes and the database SELECT.

Task 3 — Analyze /DMO/CONNECTION

Open the ABAP Development Object search with Ctrl + Shift + A. Search for /dmo/connection and open /DMO/CONNECTION (Database Table).

The relevant fields are:

Field	ABAP Type	Meaning
airport_from_id	/DMO/AIRPORT_FROM_ID	Departure / From airport
airport_to_id	/DMO/AIRPORT_TO_ID	Destination / To airport

The database table also contains the key fields carrier_id and connection_id. We will use those values in the WHERE condition to find the correct row.

Why do we normally not specify the client field? ABAP Open SQL performs client handling automatically for client-dependent tables, so the application normally does not need to put the client field into this WHERE condition.

Task 4 — Read Data from the Database

The constructor is used because an object should be initialized when it is created. The constructor receives i_carrier_id and i_connection_id, so those values can be used immediately to locate the matching database record.

Step 4.1 — Validate the constructor input

```
IF i_carrier_id IS INITIAL OR i_connection_id IS INITIAL.
  RAISE EXCEPTION TYPE cx_abap_invalid_value.
ENDIF.
```

If either key is empty, there is no valid connection to look up. The constructor therefore raises CX_ABAP_INVALID_VALUE before accessing the database.

Step 4.2 — Write SELECT SINGLE

```
SELECT SINGLE
  FROM /dmo/connection
  FIELDS airport_from_id, airport_to_id
  WHERE carrier_id = @i_carrier_id
    AND connection_id = @i_connection_id
  INTO ( @airport_from_id, @airport_to_id ).
```

Understand the statement

Part	Meaning	Reason
SELECT SINGLE	Read one matching row	One carrier/connection combination identifies one connection
FROM /dmo/connection	Database table	This table stores the connection data
FIELDS airport_from_id, airport_to_id	Columns to read	Only the required airport values are selected
WHERE carrier_id = @i_carrier_id	Match carrier	Uses constructor input as database key
AND connection_id = @i_connection_id	Match connection	Completes the key selection
INTO (@airport_from_id, @airport_to_id)	Receive the result	Stores DB values in private object attributes

Why the @ symbol?

The @ marks an ABAP host variable inside Open SQL. carrier_id and connection_id in the WHERE clause are database columns; i_carrier_id and i_connection_id are ABAP constructor parameters. Therefore the parameters need @.

```
WHERE carrier_id = @i_carrier_id
```

Without @, the statement would not clearly identify the expression as an ABAP host variable in modern Open SQL syntax.

Step 4.3 — Check SY-SUBRC

```
IF sy-subrc <> 0.
  RAISE EXCEPTION TYPE cx_abap_invalid_value.
ENDIF.
```

SY-SUBRC is the system return code. After SELECT SINGLE, a value of 0 indicates success. A non-zero value means the expected row was not found. The exercise converts that failure into CX_ABAP_INVALID_VALUE so the caller's TRY...CATCH can handle it.

Step 4.4 — Constructor in full

```
METHOD constructor.

  IF i_carrier_id IS INITIAL OR i_connection_id IS INITIAL.
    RAISE EXCEPTION TYPE cx_abap_invalid_value.
  ENDIF.

  SELECT SINGLE
    FROM /dmo/connection
    FIELDS airport_from_id, airport_to_id
    WHERE carrier_id = @i_carrier_id
      AND connection_id = @i_connection_id
    INTO ( @airport_from_id, @airport_to_id ).

  IF sy-subrc <> 0.
    RAISE EXCEPTION TYPE cx_abap_invalid_value.
  ENDIF.

  me->carrier_id = i_carrier_id.
  me->connection_id = i_connection_id.

  conn_counter = conn_counter + 1.

ENDMETHOD.
```

Complete Exercise 12 Solution

Local class:

```
CLASS lcl_connection DEFINITION.

  PUBLIC SECTION.

    CLASS-DATA conn_counter TYPE i READ-ONLY.

    METHODS constructor
      IMPORTING
        i_carrier_id    TYPE /dmo/carrier_id
        i_connection_id TYPE /dmo/connection_id
      RAISING
        cx_abap_invalid_value.

    METHODS get_output
      RETURNING
        VALUE(r_output) TYPE string_table.

  PROTECTED SECTION.

  PRIVATE SECTION.

    DATA carrier_id      TYPE /dmo/carrier_id.
    DATA connection_id   TYPE /dmo/connection_id.
    DATA airport_from_id TYPE /dmo/airport_from_id.
    DATA airport_to_id   TYPE /dmo/airport_to_id.

ENDCLASS.


CLASS lcl_connection IMPLEMENTATION.
```

```
  METHOD constructor.

    IF i_carrier_id IS INITIAL OR i_connection_id IS INITIAL.
      RAISE EXCEPTION TYPE cx_abap_invalid_value.
    ENDIF.

    SELECT SINGLE
      FROM /dmo/connection
      FIELDS airport_from_id, airport_to_id
      WHERE carrier_id = @i_carrier_id
        AND connection_id = @i_connection_id
      INTO ( @airport_from_id, @airport_to_id ).

    IF sy-subrc <> 0.
      RAISE EXCEPTION TYPE cx_abap_invalid_value.
    ENDIF.

    me->carrier_id = i_carrier_id.
    me->connection_id = i_connection_id.

    conn_counter = conn_counter + 1.

  ENDMETHOD.


  METHOD get_output.

    APPEND |------------------------------| TO r_output.
    APPEND |Carrier: { carrier_id }| TO r_output.
    APPEND |Connection: { connection_id }| TO r_output.
    APPEND |Departure: { airport_from_id }| TO r_output.
    APPEND |Destination: { airport_to_id }| TO r_output.

  ENDMETHOD.

ENDCLASS.
```

Global Class main( )

```
METHOD if_oo_adt_classrun~main.

  DATA connection TYPE REF TO lcl_connection.
  DATA connections TYPE TABLE OF REF TO lcl_connection.

  TRY.
      connection = NEW #(
        i_carrier_id    = 'LH'
        i_connection_id = '0400'
      ).
      APPEND connection TO connections.
    CATCH cx_abap_invalid_value.
      out->write( `Method call failed` ).
  ENDTRY.

  TRY.
      connection = NEW #(
        i_carrier_id    = 'AA'
        i_connection_id = '0017'
      ).
      APPEND connection TO connections.
    CATCH cx_abap_invalid_value.
      out->write( `Method call failed` ).
  ENDTRY.
```

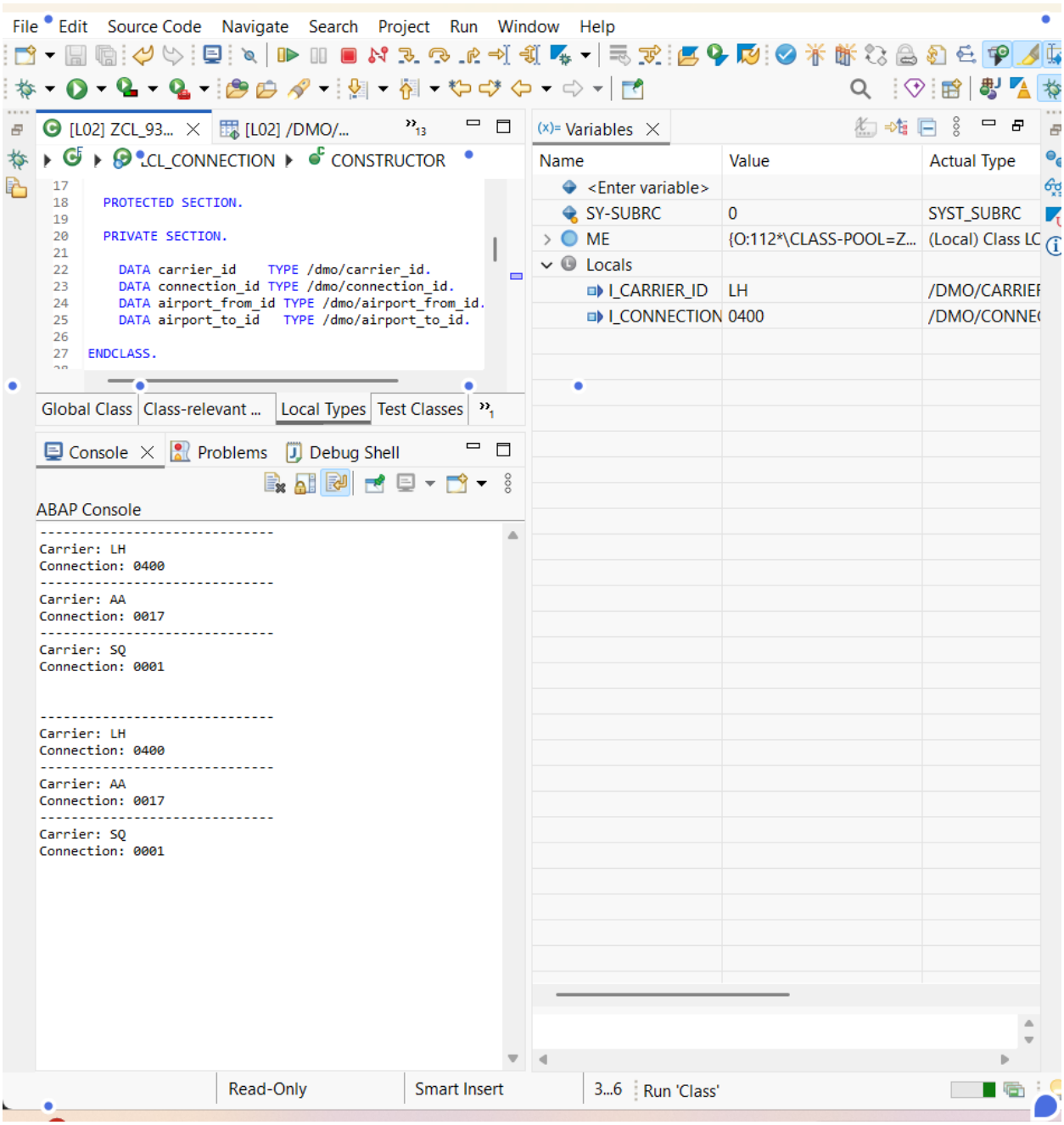
```
  TRY.
      connection = NEW #(
        i_carrier_id    = 'SQ'
        i_connection_id = '0001'
      ).
      APPEND connection TO connections.
    CATCH cx_abap_invalid_value.
      out->write( `Method call failed` ).
  ENDTRY.

  LOOP AT connections INTO connection.
    out->write( connection->get_output( ) ).
  ENDLOOP.

ENDMETHOD.
```

Debugging and Execution

Activate the class with Ctrl + F3. Run it with F9. If a breakpoint stops execution inside the constructor, inspect the Variables view. For the first object you should see I_CARRIER_ID = LH and I_CONNECTION_ID = 0400. Press F8 to continue.

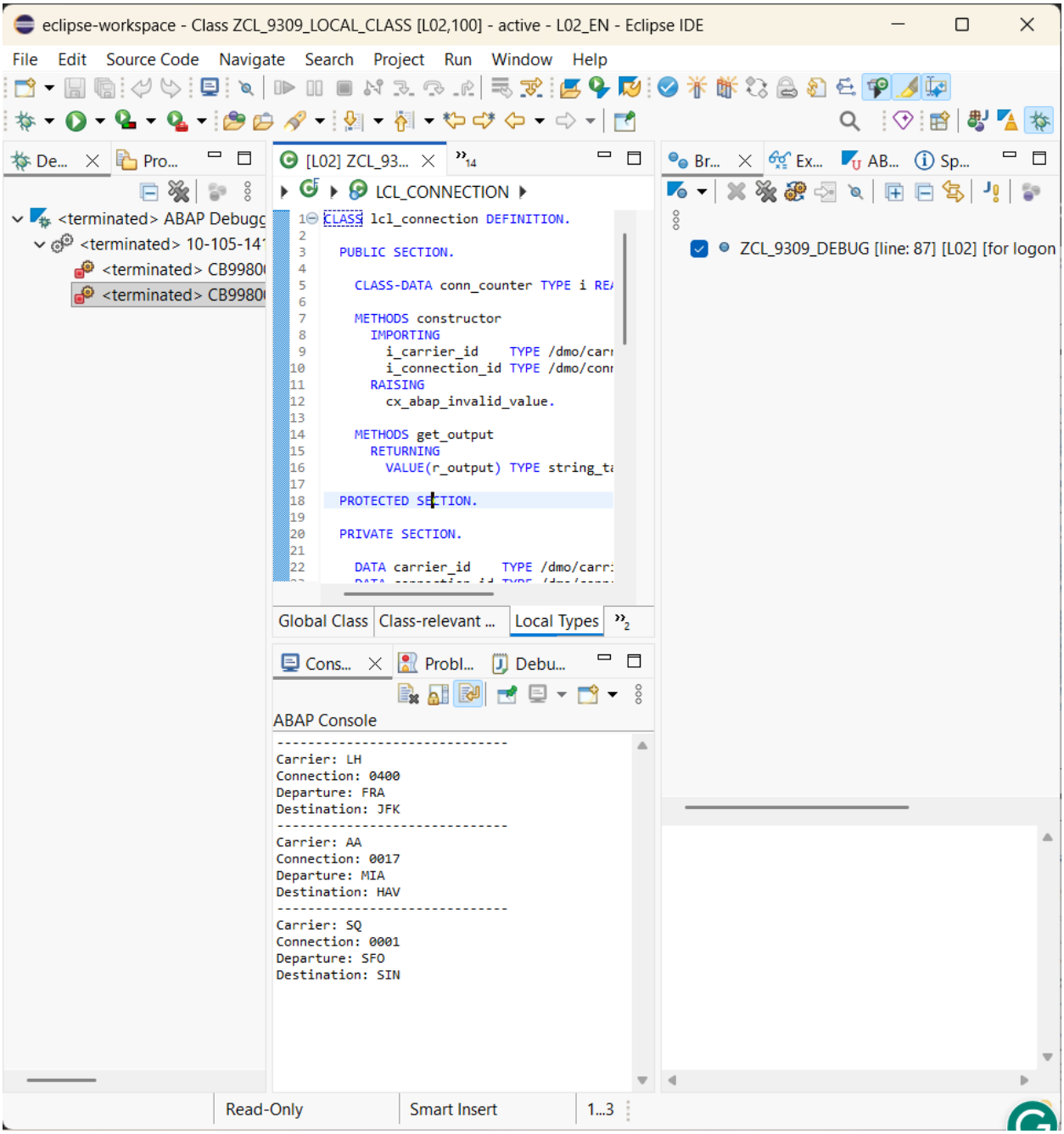


Screenshot 2 — Debugger showing constructor parameters for the LH / 0400 instance.

Expected successful output

In the completed run from your practice, the database lookup returned the following values:

Carrier	Connection	Departure	Destination
LH	0400	FRA	JFK
AA	0017	MIA	HAV
SQ	0001	SFO	SIN



Screenshot 3 — Final ABAP Console output from the completed Exercise 12 run.

How the final result was produced

```
LH / 0400
   |
   +--> SELECT /DMO/CONNECTION
   |
   +--> airport_from_id = FRA
   +--> airport_to_id   = JFK
   |
   +--> object attributes
   |
   +--> get_output( )
   |
   +--> ABAP Console
```

The same process occurs for AA / 0017 and SQ / 0001. The important point is that FRA, JFK, MIA, HAV, SFO, and SIN were not hard-coded into get_output( ). They were read from the database.

Troubleshooting

Problem	What to check
Departure/Destination blank	Activate with Ctrl+F3 and run again. Make sure the SELECT is in the constructor.
SELECT syntax error	Check FROM, FIELDS, WHERE, @ host variables, and INTO exactly.
SY-SUBRC is not 0	The carrier/connection combination has no matching row in /DMO/CONNECTION.
Debugger keeps stopping	A breakpoint is active. Press F8 or disable the unnecessary breakpoint.
Cannot find lcl_connection	Open your own ZCL_... class and select the Local Types tab.

Key concepts to remember

- Constructor: initializes the object during NEW.
- Private attributes: internal state controlled by the class.
- SELECT SINGLE: reads one matching database row.
- FIELDS: selects only the database columns required.
- WHERE: identifies the required database row.
- @: identifies an ABAP host variable in Open SQL.
- INTO: receives the selected values into ABAP variables/attributes.
- SY-SUBRC: checks whether the database operation succeeded.
- CX_ABAP_INVALID_VALUE: exception used by this exercise for invalid/missing data.

Exercise 12 Checklist

- Added airport_from_id privately.
- Added airport_to_id privately.
- Displayed both attributes in get_output( ).
- Inspected /DMO/CONNECTION.
- Confirmed airport_from_id and airport_to_id.
- Added SELECT SINGLE.
- Used carrier_id and connection_id in WHERE.
- Used @ for constructor parameters.
- Stored results in private attributes.
- Checked SY-SUBRC.
- Activated and executed the class.
- Verified real airport values in the console.

Exercise 12 — Completed Successfully